The selector syntax that have_selector uses is very similar to jQuery and other DOM selection tools. As in jQuery, the use of two separate selectors means you'd expect to match an instance of the first selector, which contains an instance of the second. For example, you're looking for an HTML element with the DOM ID of the form project_12, where 12 is the on_schedule project's ID. You also need that outer HTML element to contain an inner HTML element with a DOM class on_schedule.

The pattern project_12 is exactly what the Rails dom_id helper uses, and you previously put that in the tr element of each project in the index listing. So you're looking for an on_schedule class inside that view.

This test will fail with an error message that looks something like this, showing that the test is looking for a pattern that is not found:

```
Failure/Error:
       expect(rendered).to have_selector(
           "#project_#{on_schedule.id} .on_schedule")

       expected to find visible css "#project_1 .on_schedule" but there
       were no matches
```

A minor change in your index template gets the test to pass—change the project-name cell to use the name_with_status helper:

display/04/app/views/projects/index.html.erb

```
<h1>All Projects</h1>
<table>
  <thead>
    <tr>
      <td>Project Name</td>
      <td>Total Project Size</td>
    </tr>
  </thead>
  <tbody>
    <% @projects.each do |project| %>
      <tr class="project-row" id="<%= dom_id(project) %>">
        <td class="name"><%= name_with_status(project) %></td>
        <td class="total-size"><%= project.size %></td>
      </tr>
    <% end %>
  </tbody>
</table>
```

Now the tr field with the appropriate DOM ID has a span element from the helper that contains the expected DOM class.